

Strongly Consistent Mutable Global Indexes

Kadir Ozdemir
October 2019



Team



- Geoffrey Jacoby
- Gokcen Iskender
- Kadir Ozdemir
- Priyank Porwal
- Swaroopa Kadam



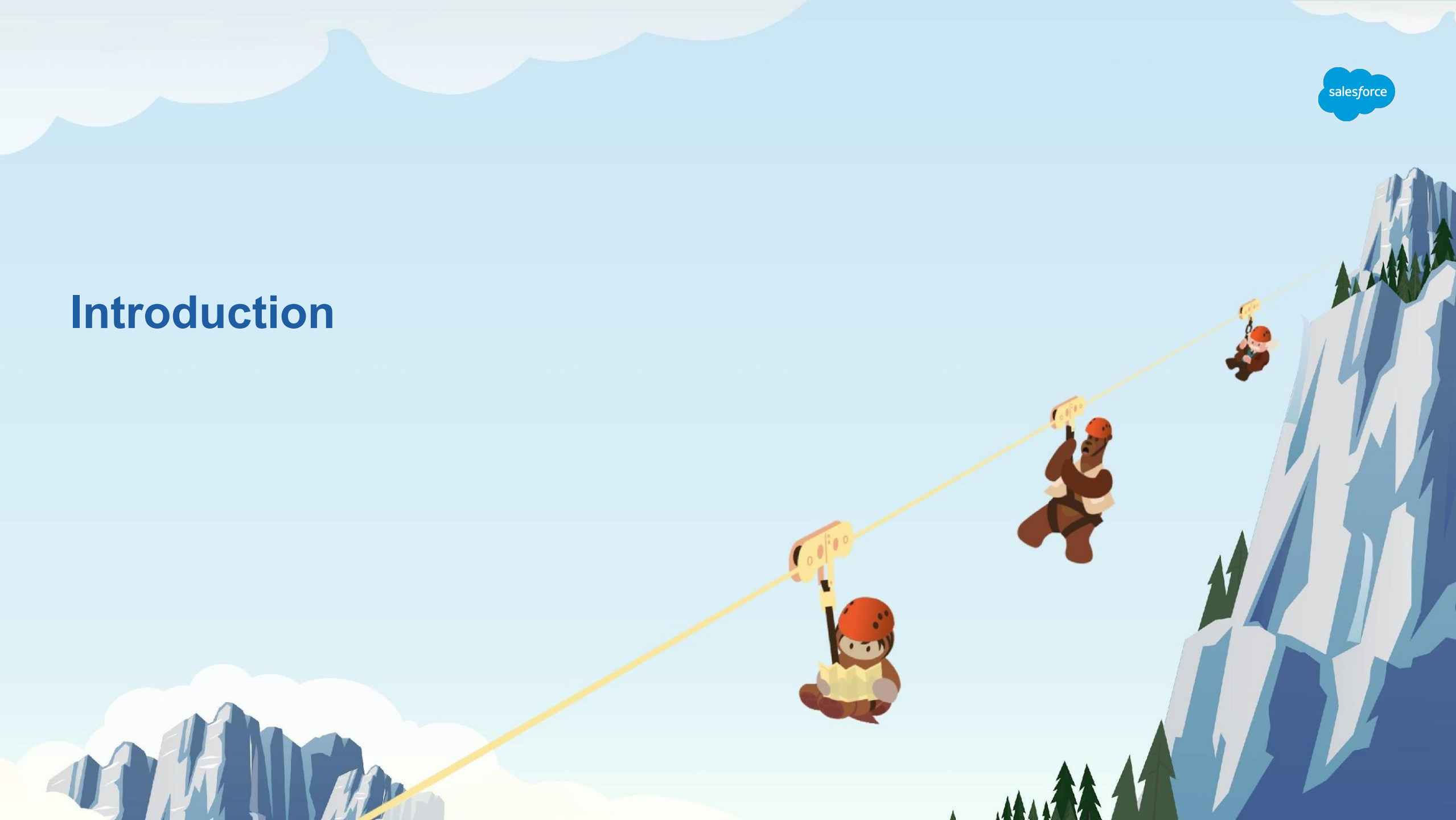
Agenda



- Quick introduction to Phoenix and secondary indexing
- Current mutable global secondary index design
- Strongly consistent mutable global secondary index design
- Correctness of the design
- Performance impact of strong consistency
- Demo for failure handling
- Code walkthrough



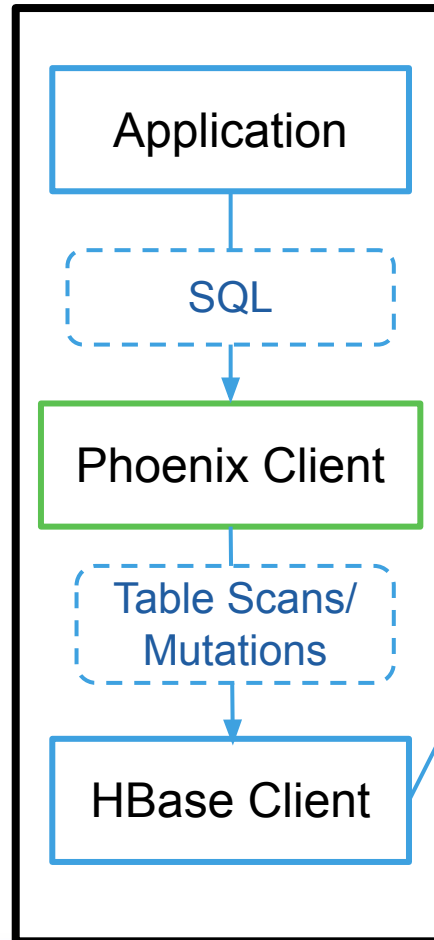
Introduction



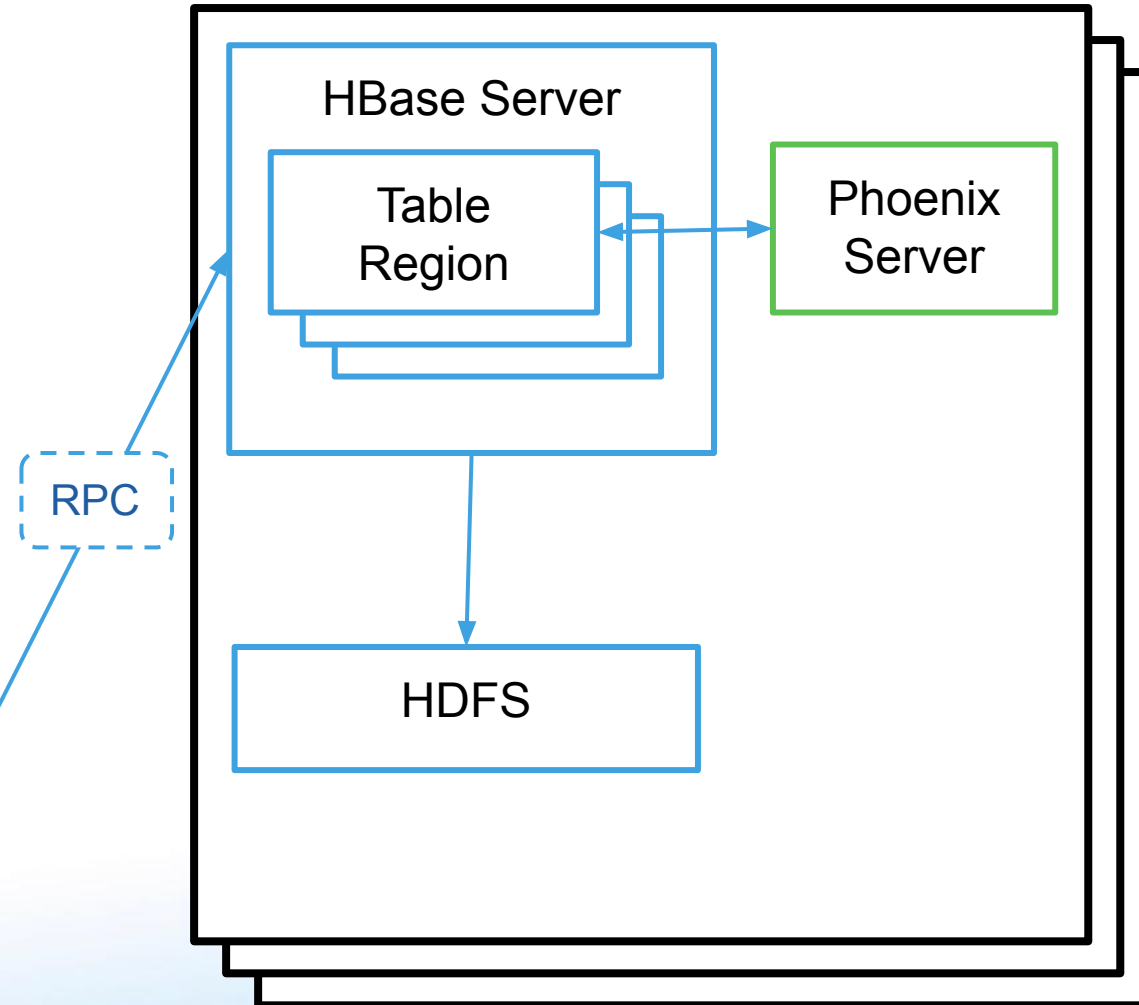
Phoenix



Application Server



HBase Region Servers



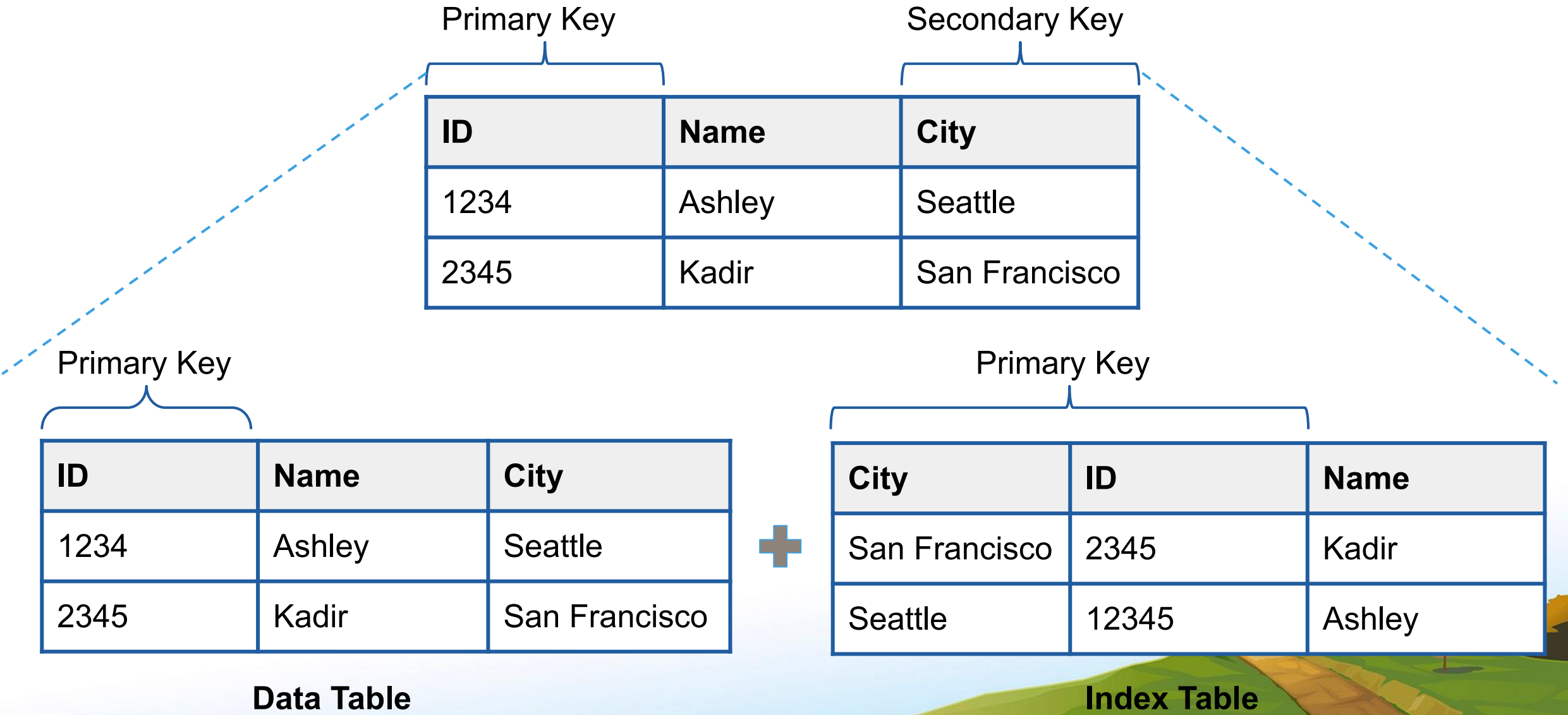
Secondary Indexing



Primary Key		Secondary Key	
ID	Name	City	
1234	Ashley	Seattle	
2345	Kadir	San Francisco	



Secondary Indexing



Secondary Indexing - Update



Primary Key

ID	Name	City
1234	Ashley	Seattle

ID	Name	City
2345	Kadir	San Francisco

Data Table

Primary Key

City	ID	Name
San Francisco	2345	Kadir

City	ID	Name
Seattle	12345	Ashley

Index Table

Secondary Indexing - Update



Primary Key

ID	Name	City
1234	Ashley	Seattle

ID	Name	City
2345	Kadir	San Francisco

Data Table

Primary Key

City	ID	Name

City	ID	Name
Seattle	12345	Ashley

Index Table

Global Secondary Indexing - Update



Primary Key

ID	Name	City
1234	Ashley	Seattle

ID	Name	City
2345	Kadir	Seattle

Data Table

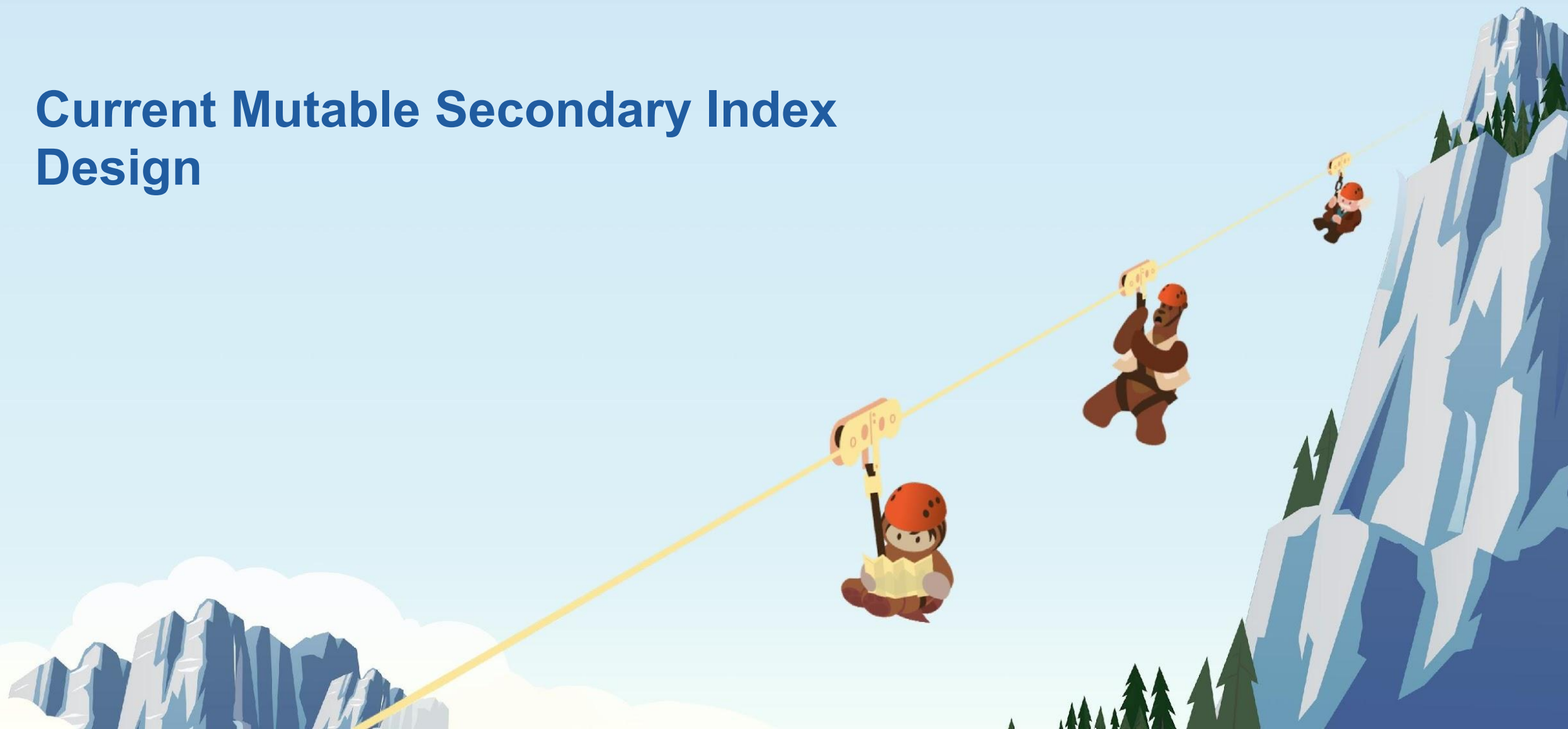
Primary Key

City	ID	Name

City	ID	Name
Seattle	1234	Ashley
Seattle	2345	Kadir

Index Table

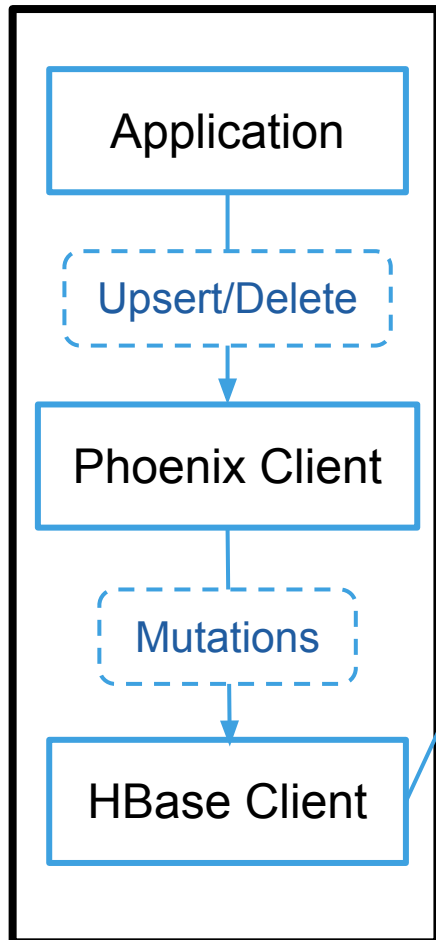
Current Mutable Secondary Index Design



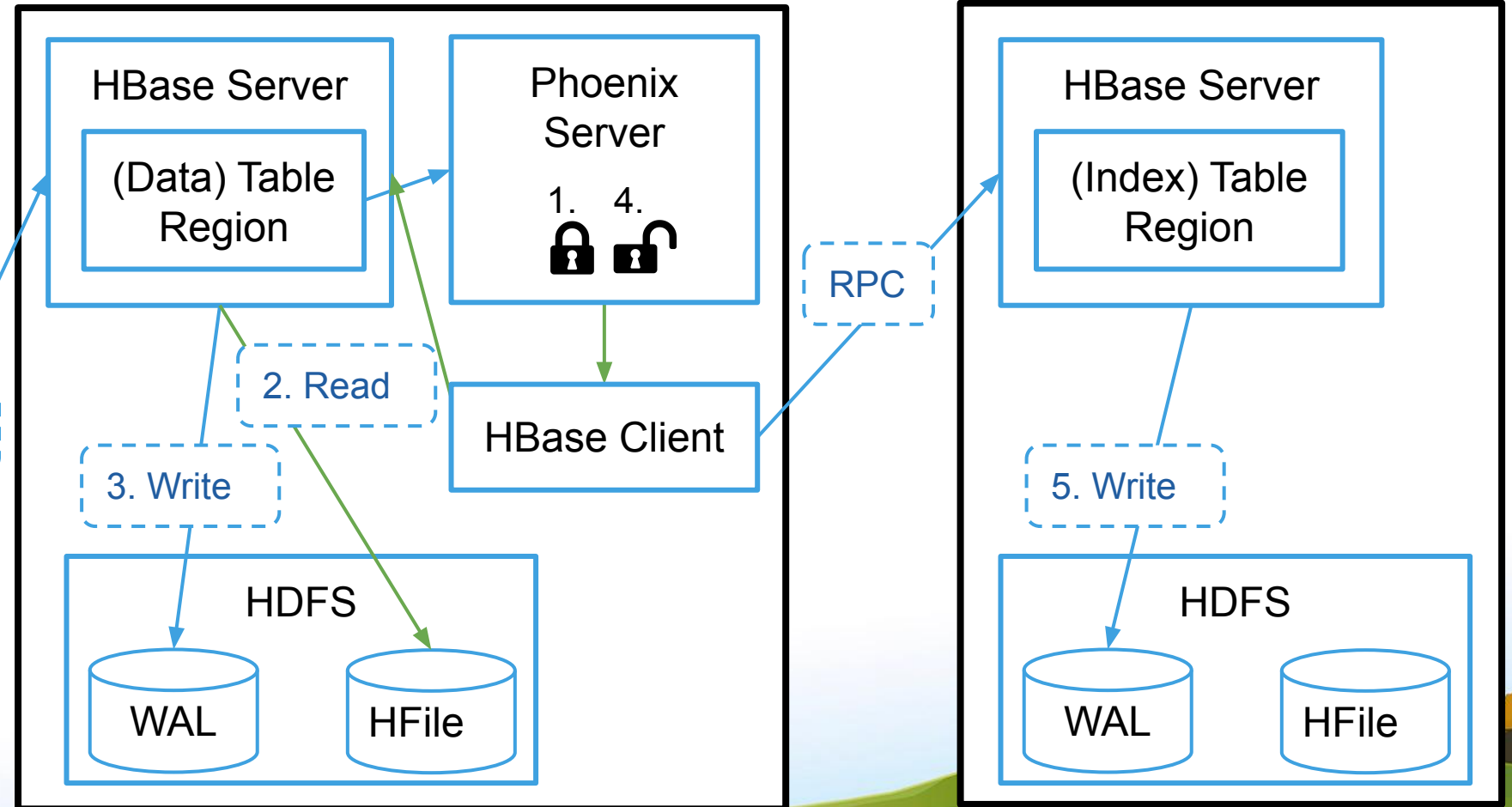
Current Design - Update Path



Application Server



HBase Region Servers



Current Design Challenges



- Tries to make tables consistent at the write time by relying on client retries
 - May not handle correlated failures and may leave data table inconsistent with its indexes
- Needs external tools to detect inconsistencies and repair them



Strongly Consistent Mutable Index Design



Design Objectives for Strongly Consistent Secondary Indexes

- Secondary indexes should be always in sync with their data tables
- Strong consistency should not result in significant performance impact
- Strong consistency should not impact scalability significantly



Observations



- Data must be consistent at read time
 - An index table row can be repaired from the corresponding data table row at read time
- In HBase writes are fast
 - We can add extra write phase without severely impacting write performance



Strongly Consistent Design



Operation	Strongly Consistent Design
Read	1. Read the index rows and check their status 2. The unverified rows repaired from the data table



Strongly Consistent Design



Operation	Strongly Consistent Design
Read	1. Read the index rows and check their status 2. The unverified rows repaired from the data table
Write	1. Set the status of existing index rows unverified and write the new index rows with the unverified status 2. Write the data table rows 3. Delete the existing index rows and set the status of new rows to verified



Strongly Consistent Design

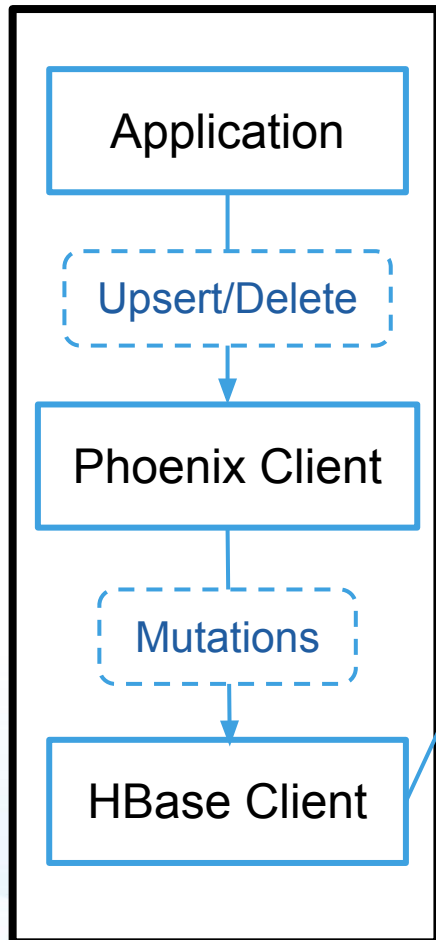


Operation	Strongly Consistent Design
Read	1. Read the index rows and check their status 2. The unverified rows repaired from the data table
Write	1. Set the status of existing index rows unverified and write the new index rows with the unverified status 2. Write the data table rows 3. Delete the existing index rows and set the status of new rows to verified
Delete	1. Set the index table rows with the unverified status 2. Delete the data table rows 3. Delete index table rows

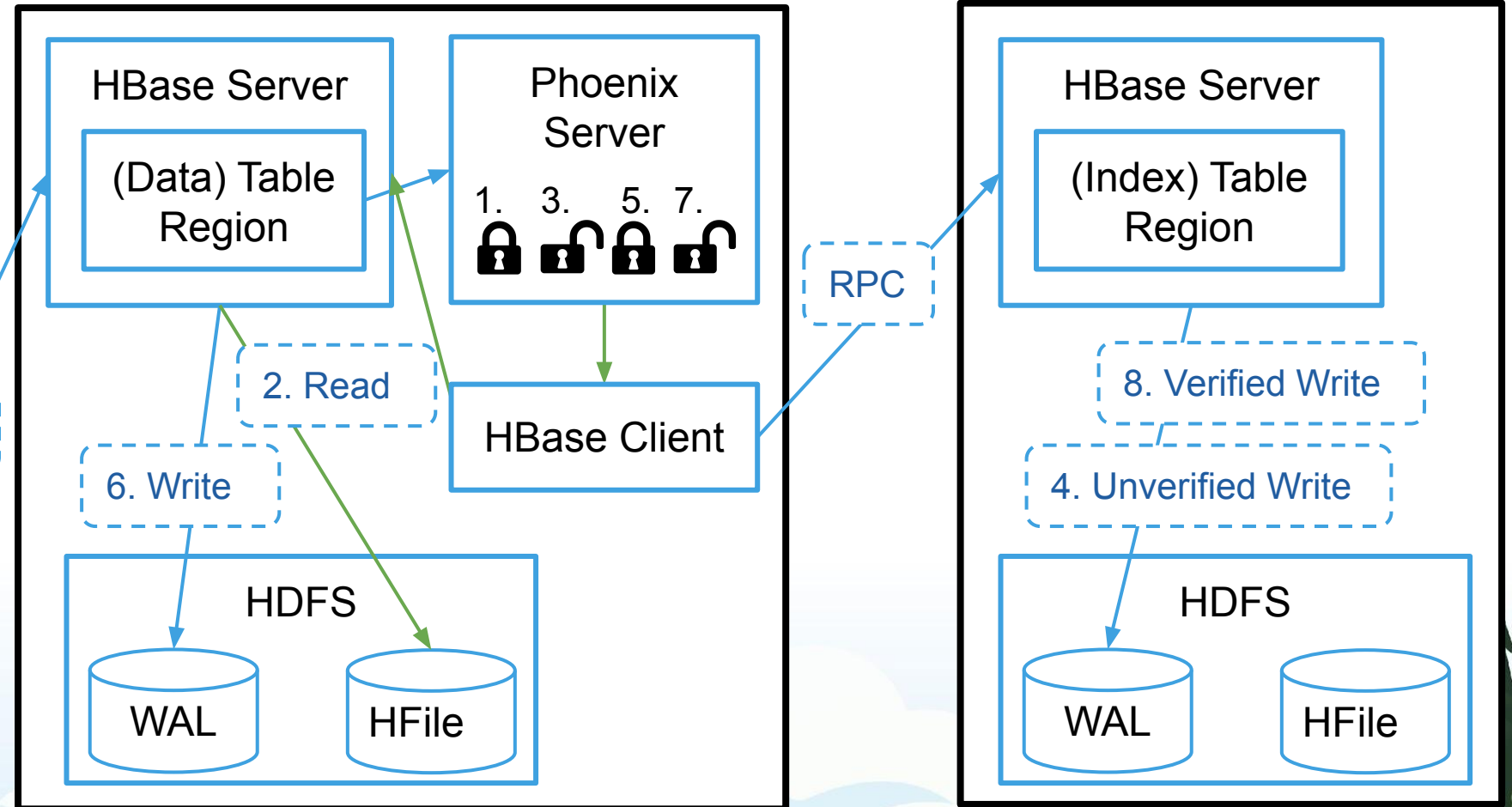
Strongly Consistent Design - Update Path



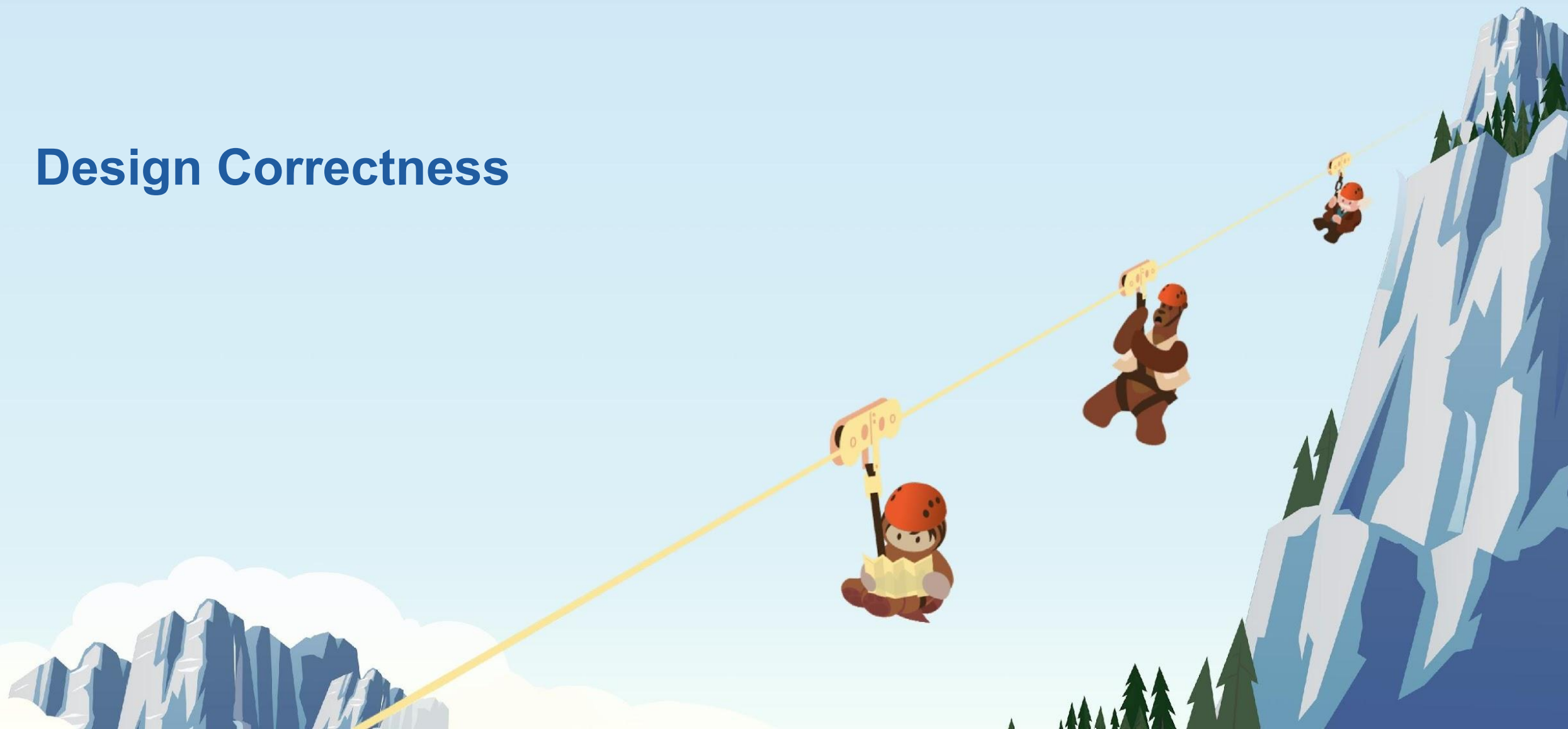
Application Server



HBase Region Servers



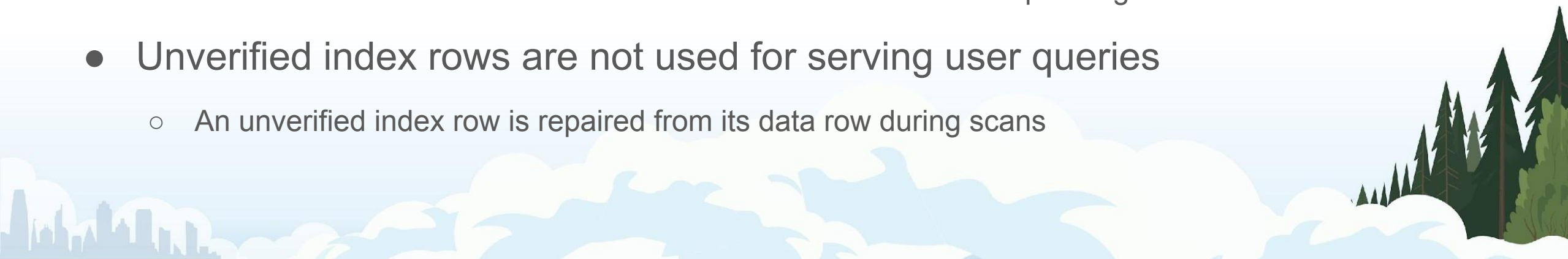
Design Correctness



Correctness Without Concurrent Row Updates



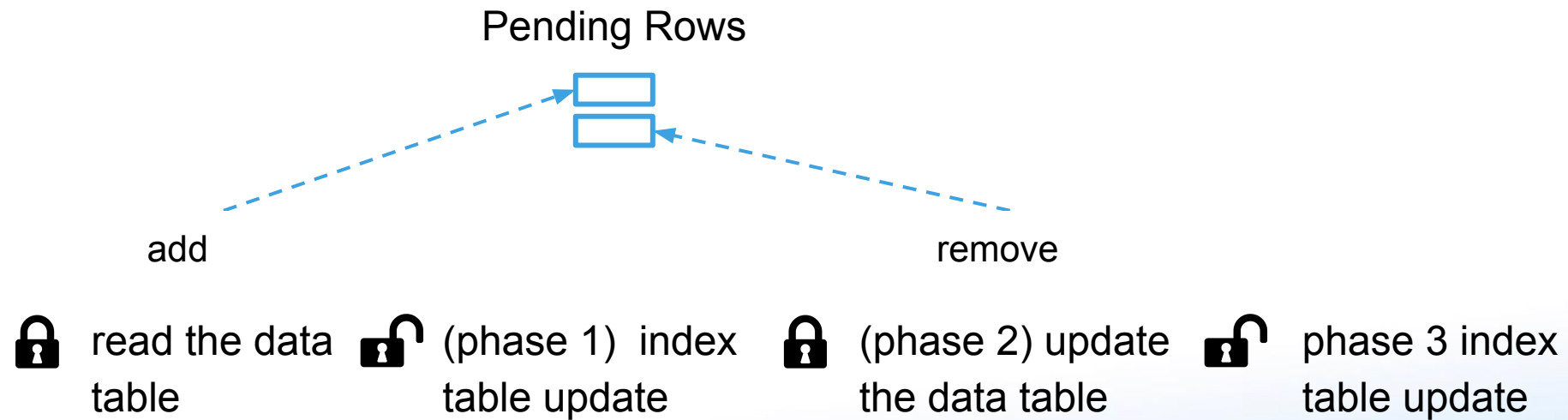
- Missing index row is not possible
 - An index row is updated first before its data row
 - If the index update is failed then the data row update will not be attempted
 - An index row is deleted only after its data table row is deleted
- Verified index row implies existence of the corresponding data row
 - The status for an index row is set to verified only after the corresponding data row is written
 - The status for an index row is set to unverified before the corresponding data row is deleted
- Unverified index rows are not used for serving user queries
 - An unverified index row is repaired from its data row during scans



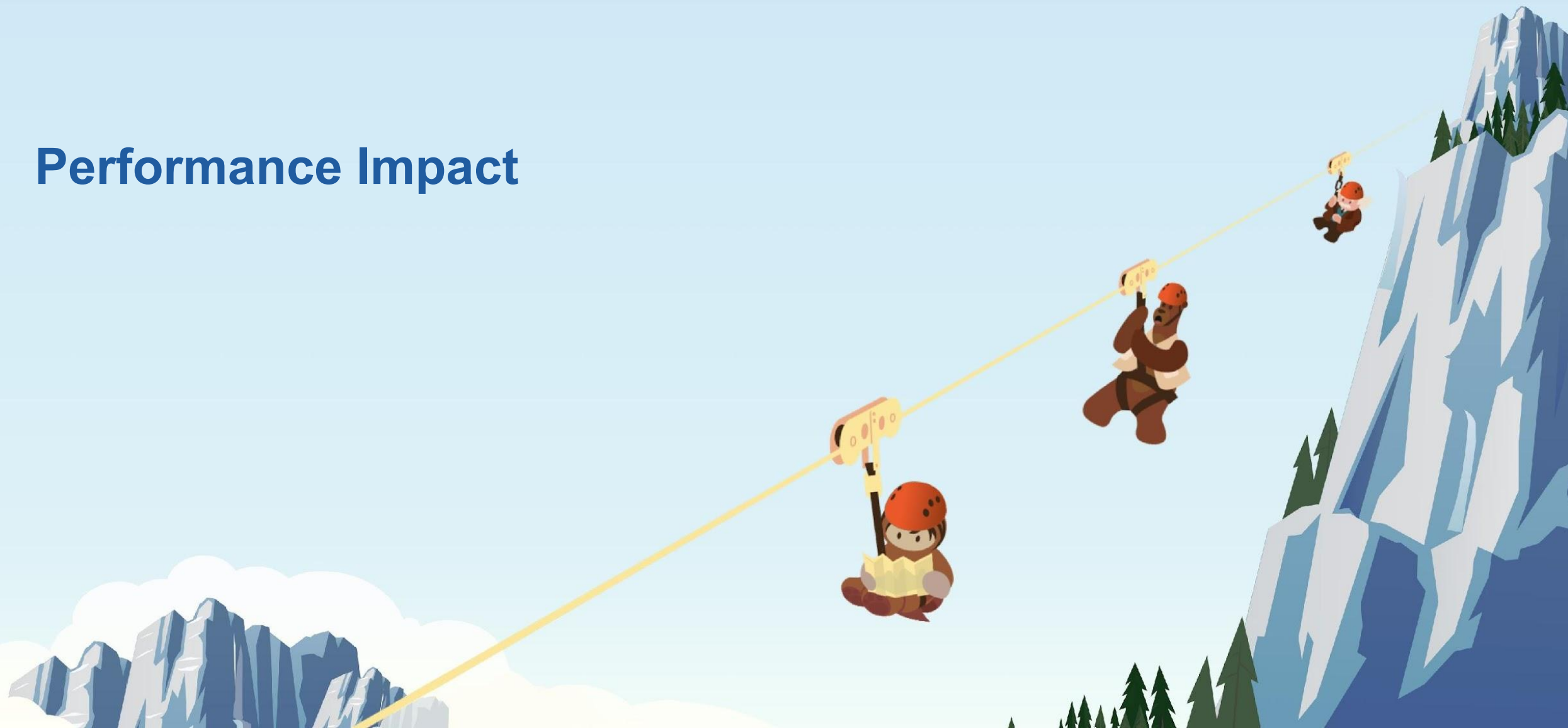
Correctness With Concurrent Row Updates



- The third phase is skipped for concurrent updates
 - Detect concurrent updates and leave them in the unverified state
- Use two phase row locking to detect concurrent updates on a data row



Performance Impact



Performance Impact of Strong Consistency



- Setup: A data table with two indexes on a 10 node cluster
 - 1 billion large rows with random primary key
 - Top N queries on indexes where N is 50
- Less than 25% increase in write latency
 - Due to setting row status in phase 3
- No noticeable increase in read latency
 - The number of unverified rows due to pending updates on a given table region is limited by the number of RPC threads and mutation batch size



Demo for Failure Handling



Tables



- CREATE TABLE CONTACTS (ID INTEGER NOT NULL PRIMARY KEY, NAME VARCHAR, ZIP INTEGER);
 - CREATE INDEX NAME_INDEX ON CONTACTS (NAME) INCLUDE (ZIP);
 - UPSERT INTO CONTACTS (ID, NAME, ZIP) VALUES(1 , 'Kadir', 94105);
 - UPSERT INTO CONTACTS (ID, NAME, ZIP) VALUES(2 , 'Gokcen', 98004);
 - Select * from CONTACTS;
- Select * from NAME_INDEX;

ID	Name	ZIP
1	Kadir	94105
2	Gokcen	98004

Name	ID	ZIP
Gokcen	2	98004
Kadir	1	94105

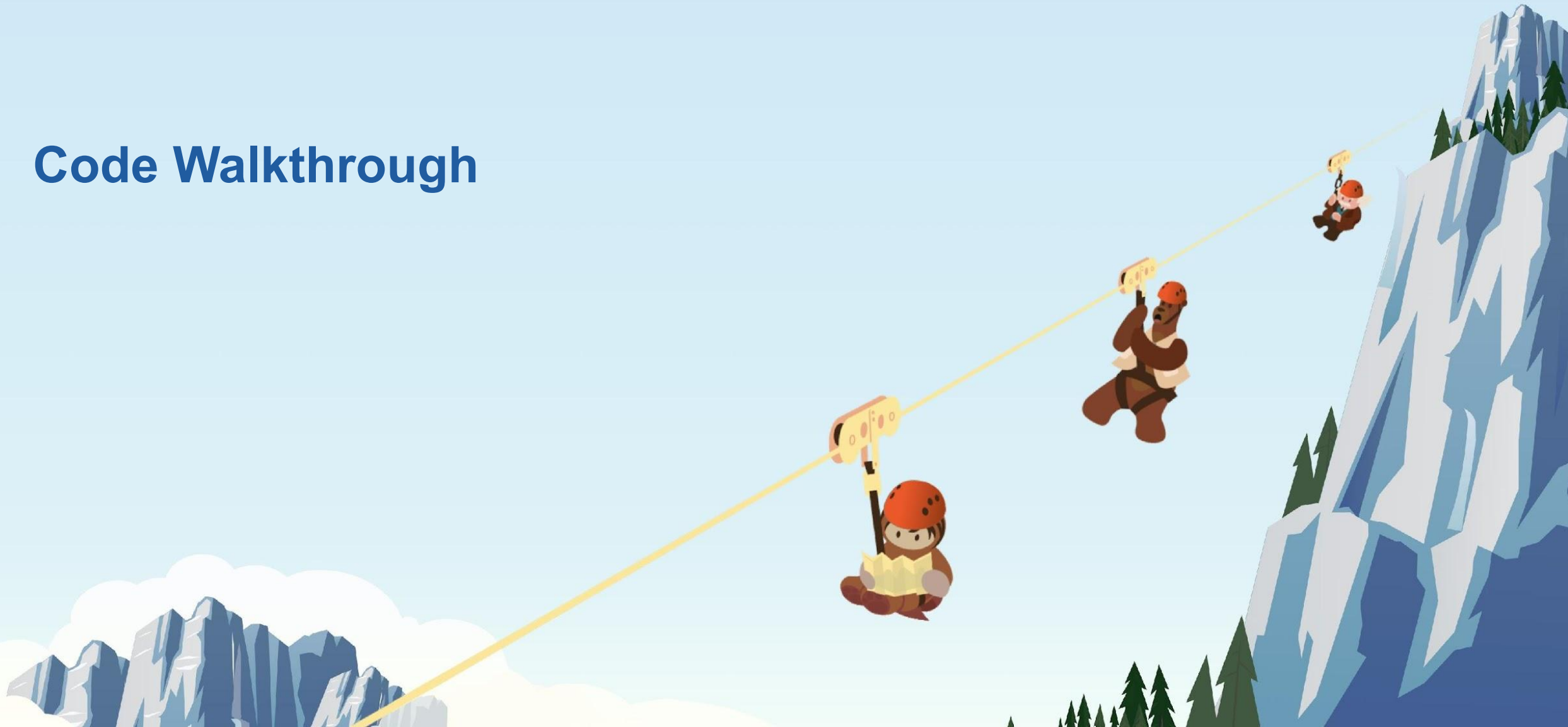
Failures



- Second write phase (i.e., data table update) failure
 - Expected behavior: The index table should not return the row
- Third write phase (i.e., post index write) failure
 - Expected behavior: The index table should read repair and return the row



Code Walkthrough



Main Classes



- IndexRegionObserver (Server)
 - Implements write path for mutable indexes
- GlobalIndexChecker (Server)
 - Implements read repair
- MutationState (Client)
 - Implement write path for immutable indexes
- IndexUtil (Client)
 - Sets the scan attributes for read repair (setScanAttributesForIndexReadRepair)





THANK YOU

Questions?